

```
'ISO-8859-1'
>>> r.text
'<!doctype html><html itemscope=""...
```

It's that easy. To make a request, simply use the method that corresponds to the verb you want to use. So to make a GET request, use `requests.get()`; to make a POST request, use `requests.post()` and so on. 'Requests' currently supports GET, POST, HEAD, OPTIONS, DELETE and PATCH. Let's break down the above code snippet, bit by bit:

```
>>> r = requests.get("https://www.google.com")
```

`r` is now a *Response* object, which is pretty powerful. Let's see what it can give us:

```
>>> r.status_code
200
```

200 means 'OK', so the HTTP request went perfectly and the server dispatched whatever data you wanted. So let's see what kind of data you can expect to see in the response body:

```
>>> r.headers["content-type"]
'text/html; charset=ISO-8859-1'
```

Right, so it's an HTML page and it's not utf-8, but plain old ISO-8859-1. Hey, while we're at it, why not see all the headers that the server sent down? Here's the dump:

```
>>> r.headers
CaseInsensitiveDict({'
  cache-control': 'private, max-age=0',
  'content-type': 'text/html; charset=ISO-8859-1',
  'x-xss-protection': '1; mode=block',
  'server': 'gws',
  'transfer-encoding': 'chunked',
  'date': 'Sat, 23 Nov 2013 19:12:11 GMT',
  ...
})
```

So it's a special kind of a dictionary - a case-insensitive one. The HTTP specification says that HTTP headers can be case insensitive, hence this. It means you can do the following:

```
>>> r.headers["Content-Type"]
'text/html; charset=ISO-8859-1'
```

And you can also do what follows:

```
>>> r.headers["Content-Type"]
'text/html; charset=ISO-8859-1'
```

And both work the same. Remember that the response

was encoded in ISO-8859-1. Well, I want utf-8. Requests can handle this.

```
>>> r.encoding
'ISO-8859-1'
>>> r.encoding = "utf-8"
>>> r.encoding
'utf-8'
```

And I've just re-encoded that data to utf-8 Unicode, on the fly.

Anyway, we've played around with the response. Now, it's time to see the actual data that the server beamed down to us. We can do it in one of three ways. The simplest is to simply access the response body as text:

```
>>> r.text
'<!doctype html><html itemscope=""...
```

If I'm downloading an image, there won't be any text to see. You're going to have to access the raw binary data, like this:

```
>>> r.content
b'<!doctype html><html itemscope=""...
```

Requests will automatically decompress the response if it's been encoding with *gzip* or *deflate*. `r.content` will give you the uncompressed raw byte stream.

This is all good for small data - a JSON response or a Web page. What if you're downloading a multi-gigabyte file? 'Requests' supports streaming responses, so you can do the following:

```
>>> r = requests.get("http://example.com/really-big-file.bin")
>>> with open("local-file.bin", "wb") as fd:
...     for chunk in r.iter_content(chunk_size):
...         fd.write(chunk)
```

That'll save *really-big-file.bin* as *local-file.bin*, and since it's using HTTP streaming downloads, it won't cache the entire file in memory before dumping it into the local file.

'Requests' has an awesome trick that makes it the perfect choice when you're writing REST API clients. It can automatically decode JSON responses into Python objects, as follows:

```
>>> import requests
>>> r = requests.get('https://github.com/timeline.json')
>>> r.headers["Content-Type"]
'application/json; charset=utf-8'
>>> r.json()
[{'u'repository': {'u'open_issues': 0, u'url': 'https://github.com/...
```